

Esempi chiamate di sistema in Windows e Unix

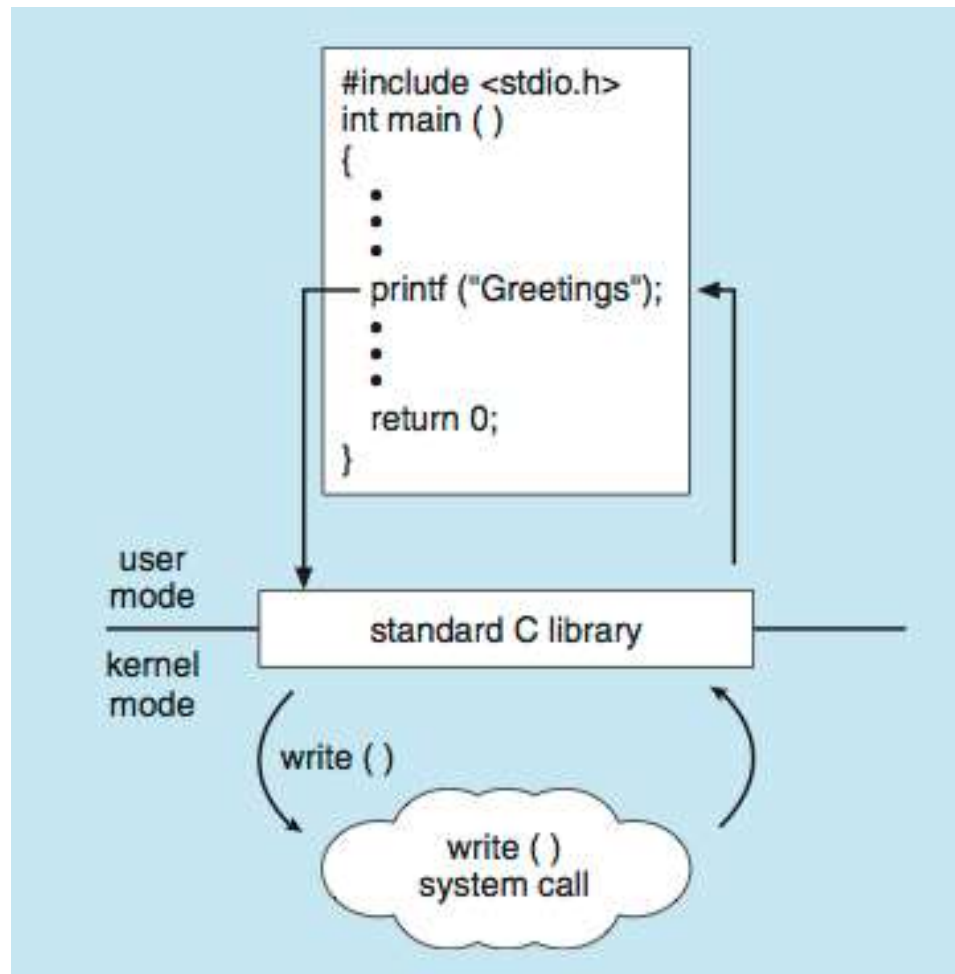


ESEMPIO DI CHIAMATE DI SISTEMA DI WINDOWS E UNIX

	Windows	UNIX
Controllo dei processi	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
Gestione dei file	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Gestione dei dispositivi	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Gestione delle informazioni	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Comunicazione	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protezione	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

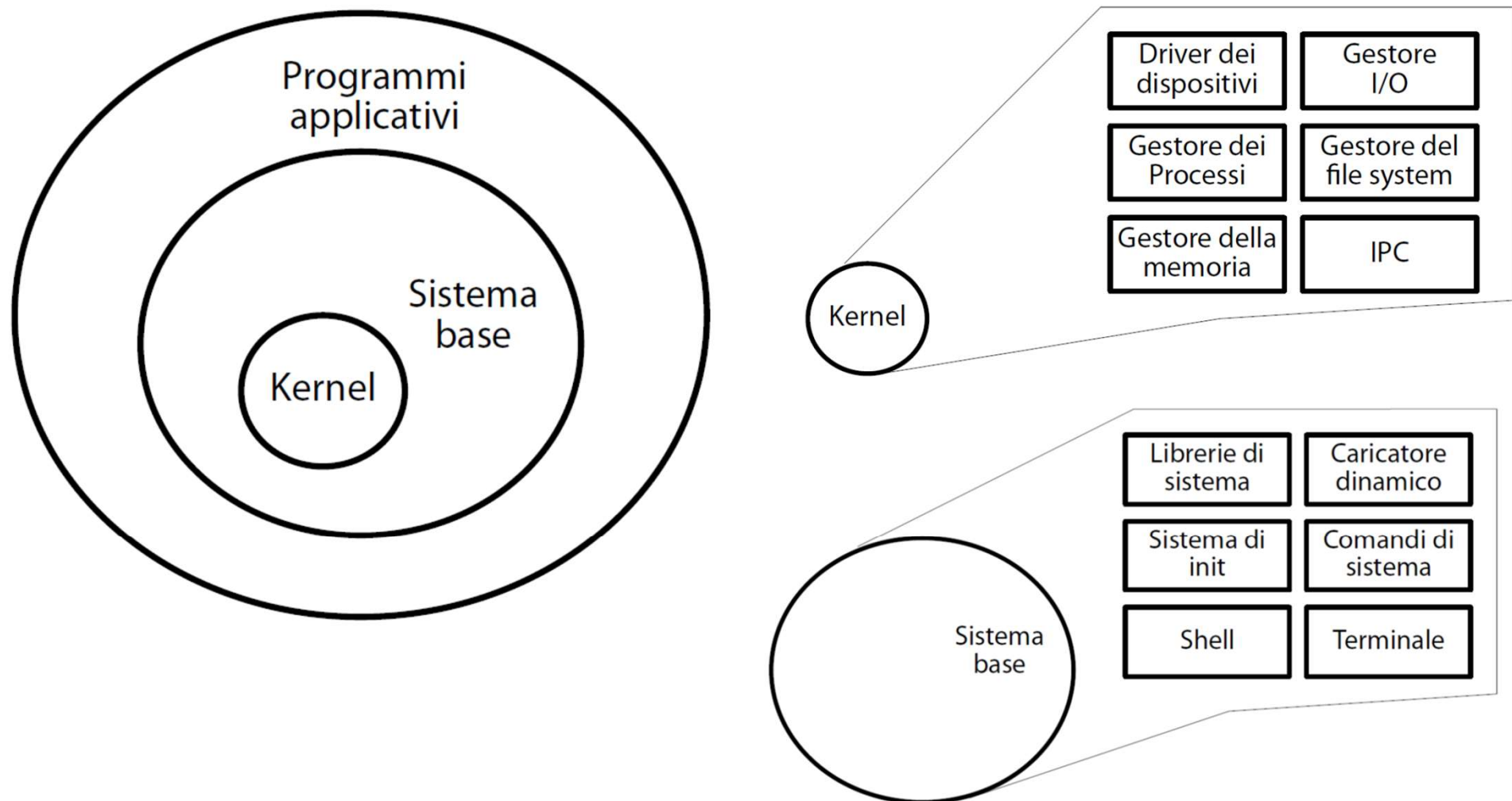
Standard C Library

- Programma in C invoca una chiamata di libreria standard `printf()` che chiama la chiamata di sistema `write()`



Programmi di Sistema

- I programmi di sistema forniscono un ambiente conveniente per lo sviluppo e l'esecuzione del programma



Programmi di Sistema

- ❑ I programmi di sistema forniscono un ambiente conveniente per lo sviluppo e l'esecuzione del programma
- ❑ Possono essere suddivisi in:
 - ❑ Manipolazione di file
 - ❑ Informazioni sullo stato
 - ❑ Supporto del linguaggio di programmazione
 - ❑ Caricamento ed esecuzione del programma
 - ❑ Comunicazioni
 - ❑ Servizi in background
 - ❑ Programmi applicativi
- ❑ La maggior parte degli utenti di un SO usa soprattutto i programmi di sistema, meno le effettive chiamate di sistema

Programmi di Sistema

- Forniscono un ambiente conveniente per lo sviluppo e l'esecuzione del programma
 - Alcuni di essi sono semplicemente interfacce utente per le chiamate di sistema; altri sono notevolmente più complessi
- **Gestione dei file:** crea, elimina, copia, rinomina, stampa, scarica, elenca e generalmente manipola file e directory
- **Informazioni sullo stato**
 - Informazioni al sistema: data, ora, quantità di memoria disponibile, spazio su disco, numero di utenti
 - Informazioni dettagliate su prestazioni, registrazione e debug
 - In genere, questi programmi formattano e stampano l'output sul terminale o su altri dispositivi di output

Programmi di Sistema

- **Modifica file**
 - Editor di testo per creare e modificare file
 - Comandi speciali per cercare contenuti di file o eseguire trasformazioni del testo
- **Supporto del linguaggio di programmazione** - compilatori, assembleri, debugger ed interpreti
- **Caricamento ed esecuzione del programma** - una volta assemblato o compilato un programma, deve essere caricato in memoria per essere eseguito. Il sistema può fornire loader, linkage editor, sistemi di debug, etc.
- **Comunicazioni** - meccanismi per creare connessioni virtuali tra processi, utenti e sistemi informatici

Programmi di Sistema

□ Servizi di Background

- Lanciati a tempo di boot
 - ▶ Alcuni per lo startup, poi terminate
 - ▶ Altri continuano a girare
- I programmi di sistema in costante esecuzioni si dicono **servizi**, **sottosistemi**, **demoni**
- Esempi sono monitoraggio di errori, servizi di stampa, etc.
- Eseguiti in contesto user, non kernel

Progettazione ed Implementazione di un SO

- La struttura di un SO può variare molto da sistema a sistema
- La progettazione parte da obiettivi e specifiche
- Dipende da hardware e tipo di sistema
- Distinguiamo **obiettivi utente** e **obiettivi di sistema**
 - Utente
 - ▶ comodo da usare, facile da imparare, affidabile, sicuro e veloce
 - Sistema
 - ▶ facile da progettare, implementare e mantenere, nonché flessibile, affidabile, privo di errori ed efficiente

Es. Windows Server vs VxWorks

Progettazione ed Implementazione di un SO

- Distinguere:
 - **Politica:** *cosa* fare?
 - **Meccanismo:** *come* fare?
 - Es., il timer serve per la protezione della CPU, il timer è un meccanismo, ma la lunghezza del timer è una politica
- La **separazione della politica dal meccanismo** consente flessibilità se le decisioni politiche devono essere modificate in un secondo momento
 - Il meccanismo deve supportare più politiche
 - La politica si può definire settando parametri dei meccanismi
 - Es., approcci a microkernel favoriscono la separazione
- Specificare e progettare un sistema operativo è un compito altamente creativo per l'ingegneria del software

Implementazione

- Diverse modalità
 - Primi SO in assembly
 - Poi linguaggi di programmazione di sistema come Algol, PL/1
 - Ora linguaggi come C, C++
- Di solito un mix di linguaggi
 - Parti più basse in assembly
 - Corpo principale in C
 - Programmi di sistema in C, C++, linguaggi di scripting come PERL, Python, shell scripts
- Più è alto il livello del linguaggio più facile è il **porting** su altro hardware, ma meno performante
- L'**emulazione** permette ad un OS di girare su hardware non-native
- La **virtualizzazione** permette l'esecuzione di più sistemi operativi

Emulazione vs Virtualizzazione

- Permette agli OS di lanciare applicazioni di altri OS
- **Emulazione**, replica software del comportamento di un'architettura diversa
 - Metodo più lento, non compilazione, ma **interpretazione**
- **Virtualizzazione** – hypervisor consente l'esecuzione a diversi sistemi operativi guest (più efficiente dell'emulatore)
 - **VMM** (Virtual Machine Manager/Monitor) fornisce servizi di virtualizzazione
 - Tipo 1:
 - ▶ VMM come Sistema operativo host, comunica direttamente con hardware (es. KVM, Oracle VR Server)
 - Tipo 2:
 - ▶ VMM installato sopra un SO host (es. VMWare, Oracle VirtualBox)

Virtualizzazione

- **Virtualizzazione** – hypervisor consente l'esecuzione a diversi sistemi operativi guest
 - **VMM** (Virtual Machine Manager/Monitor) fornisce servizi di virtualizzazione
 - Tipo 1:
 - ▶ VMM come Sistema operativo host, comunica direttamente con hardware (es. KVM, Oracle VR Server)
 - Tipo 2:
 - ▶ VMM installato sopra un SO host (es. VMWare, Oracle VirtualBox) – non emula l'HW

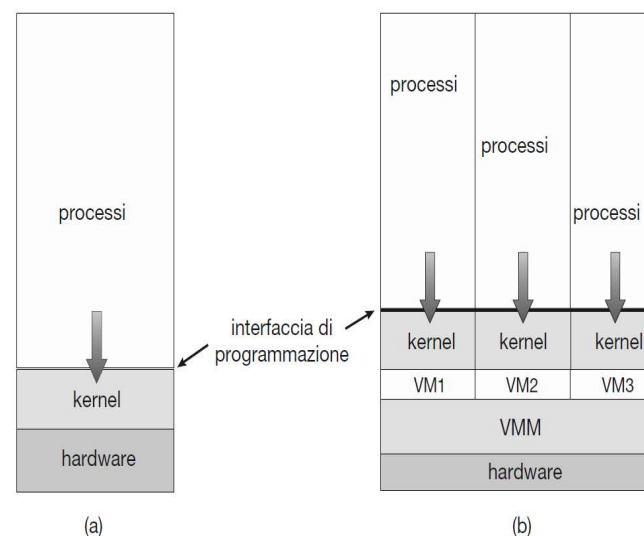


Figura 1.16 Un computer che ha in esecuzione (a) un singolo sistema operativo e (b) tre macchine virtuali.

Esempio

- **Virtualizzazione** – hypervisor consente l'esecuzione a diversi sistemi operativi guest
 - **VMM** (Virtual Machine Manager/Monitor) fornisce servizi di virtualizzazione
 - Tipo 2:
 - ▶ Oracle VirtualBox + Linux Mint
 - ▶ Comando **vmstat** ...
 - Proc: r (processi attivi), b (processi bloccati)
 - Mem: free, swap, buff
 - System: in (interrupt), cs (context switch), us (%user mode), sy (%kernel mode)
 - CPU: id (%idle time), wa (%I/O waiting time), st (%stolen time da hypervision)
 - ▶ Comando:
 - ls /proc/
 - cat /proc/interrupts
 - watch -n 1 cat /proc/interrupts

Esempio

- **Virtualizzazione** – hypervisor consente l'esecuzione a diversi sistemi operativi guest
 - **VMM** (Virtual Machine Manager/Monitor) fornisce servizi di virtualizzazione
 - Tipo 2:
 - ▶ Oracle VirtualBox + Linux Mint
 - ▶ Comando **vmstat** ...
 - Proc: r (processi attivi), b (processi bloccati)
 - Mem: free, swap, buff
 - System: in (interrupt), cs (context switch), us (%user mode), sy (%kernel mode)
 - CPU: id (%idle time), wa (%I/O waiting time), st (%stolen time da hypervision)
 - ▶ Comando:
 - cat /proc/interrupts
 - watch -n 1 cat /proc/interrupts
-
- timer: interrupt del timer di sistema
 - i8042: tastiera o il touchpad.
 - eth0: interfaccia di rete.
 - ahci: Il controller del disco
 - LOC local timer interrupt (per CPU)

IO-APIC: gestione avanzata degli interrupt nei sistemi x86

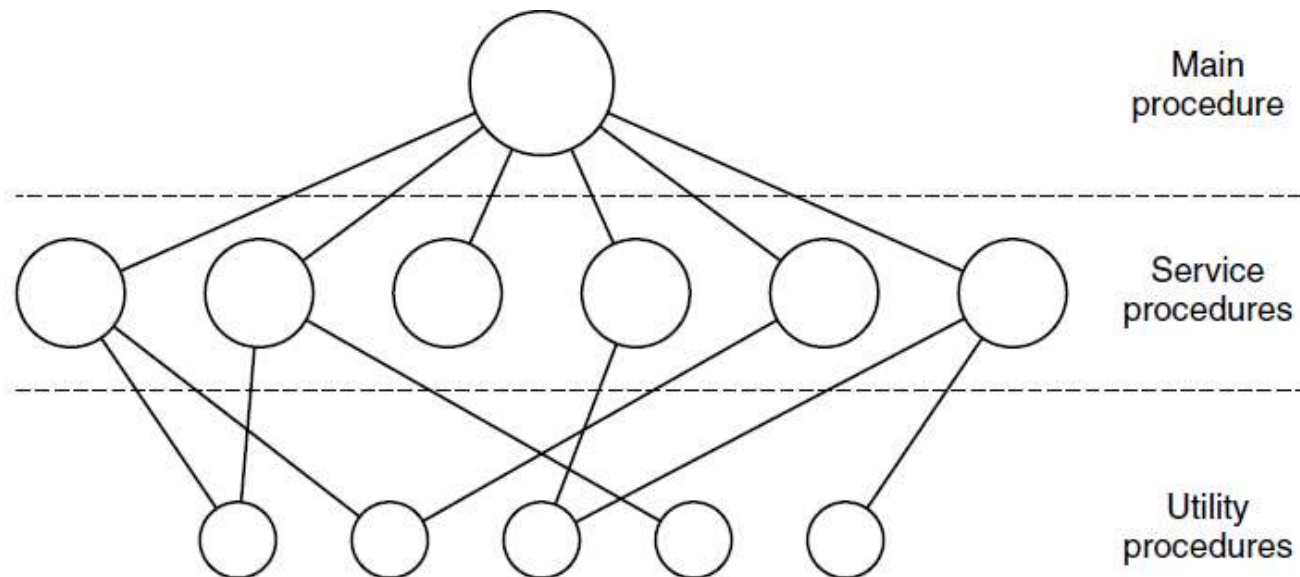
Strutture di un Sistema Operativo

- Un Sistema Operativo è un software molto complesso

- Vari modi di strutturarlo
 - Sistemi monolitici
 - ▶ Struttura semplice -- MS-DOS
 - ▶ Più complessa -- UNIX
 - Sistemi Stratificati
 - Sistemi a micro-kernel
 - Sistemi ibridi

Sistemi Monolitici

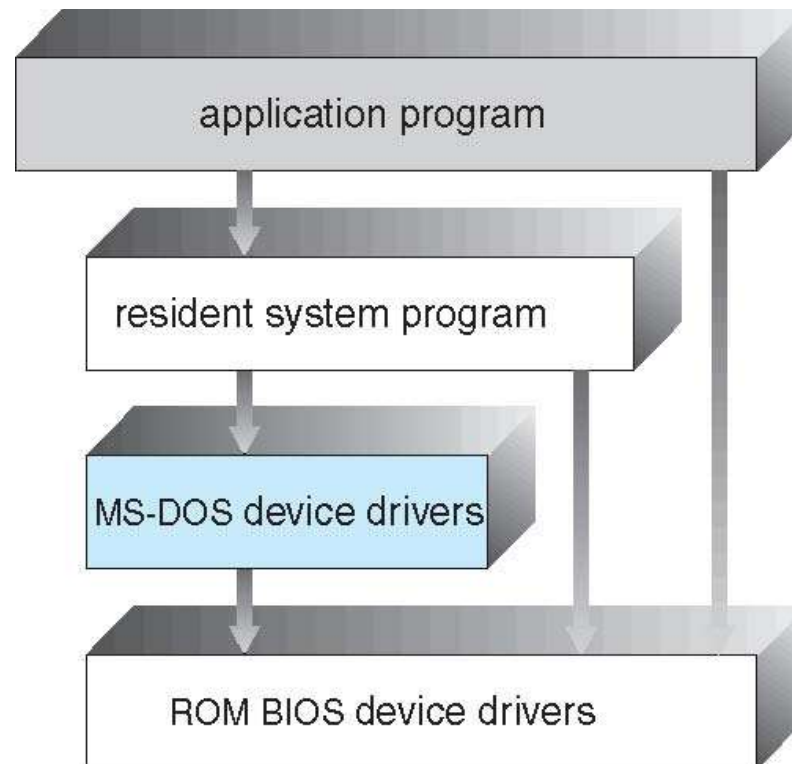
- ❑ Sistema Operativo è costituito da una collezione di procedure ognuna delle quali può chiamare qualsiasi altra
 - ❑ Un sistema monolitico viene anche chiamato sistema strettamente accoppiato (tightly coupled)
 - ▶ Collezione di procedure linkate in un unico eseguibile
 - ▶ Ogni procedura può chiamare l'altra (efficace ed efficiente)
 - ▶ Poca modularità (difficili da sviluppare ed estendere)
 - ▶ Struttura basata su chiamate di sistema



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Struttura Semplice - MS-DOS

- MS-DOS – funzionalità nel minimo spazio
 - **Monoutente e monotask** (Windows 3.1 multitask cooperativo)
 - Non suddiviso in moduli
 - Sebbene MS-DOS abbia una struttura, interfacce e livelli di funzionalità non sono ben separati
 - Programmi utente accedono a tutti i livelli (controllo completo della macchina)



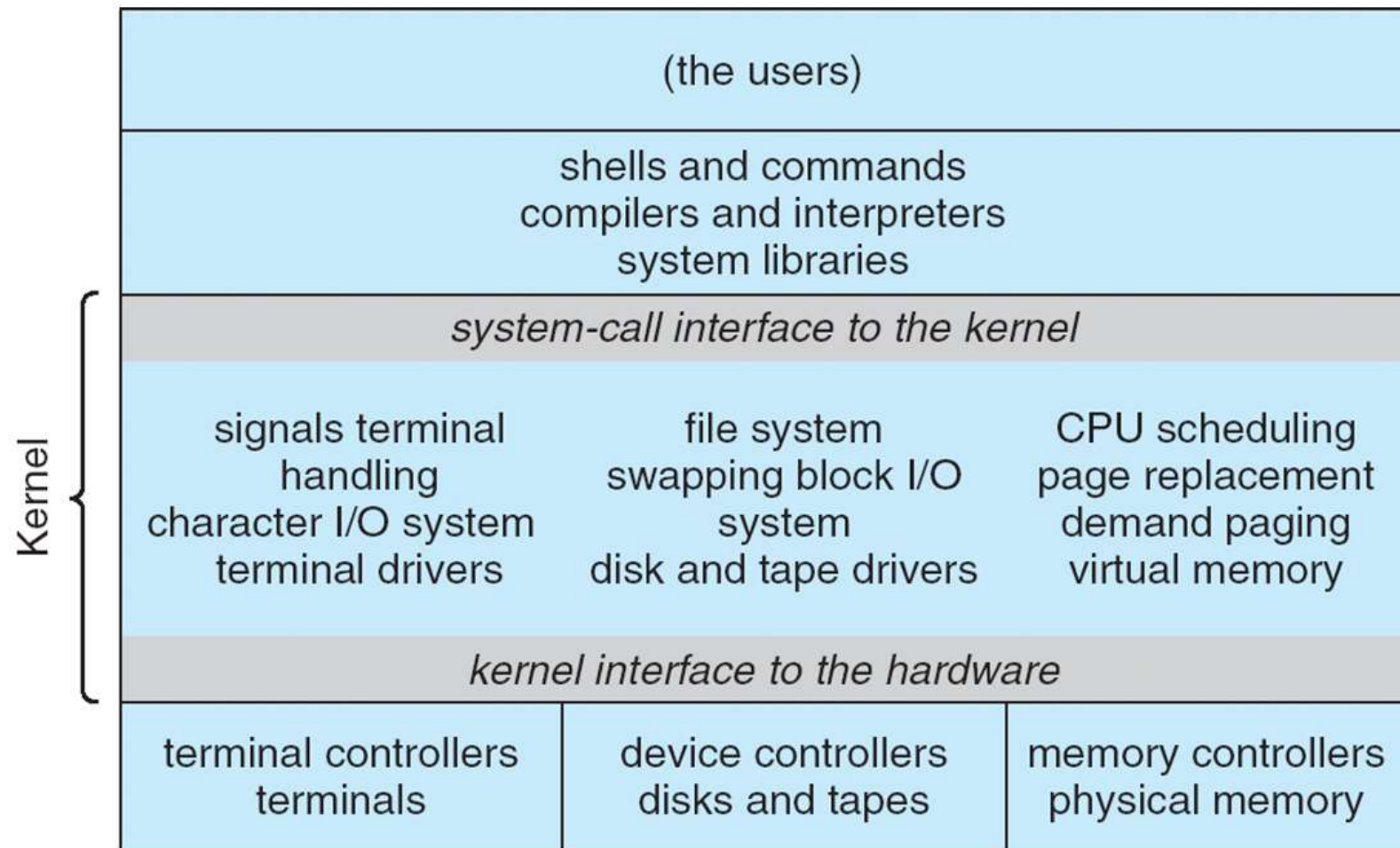
BIOS (Basic Input Output System)
avvio, interfaccia HW, interrupt

Struttura non Semplice - UNIX

- UNIX: limitato dalla funzionalità hardware, il sistema operativo UNIX originale aveva una strutturazione limitata
- Il sistema operativo UNIX è costituito da due parti separabili
 - **Programmi di sistema**
 - **Kernel**
 - ▶ Tutto ciò che si trova al di sotto dell'interfaccia di chiamata di sistema e al di sopra dell'hardware fisico
 - ▶ Fornisce il file system, schedulazione della CPU, gestione della memoria e altre funzioni del sistema operativo (alto numero di funzioni in un livello)
 - ▶ Interfaccia standard per chiamate di sistema (POSIX)

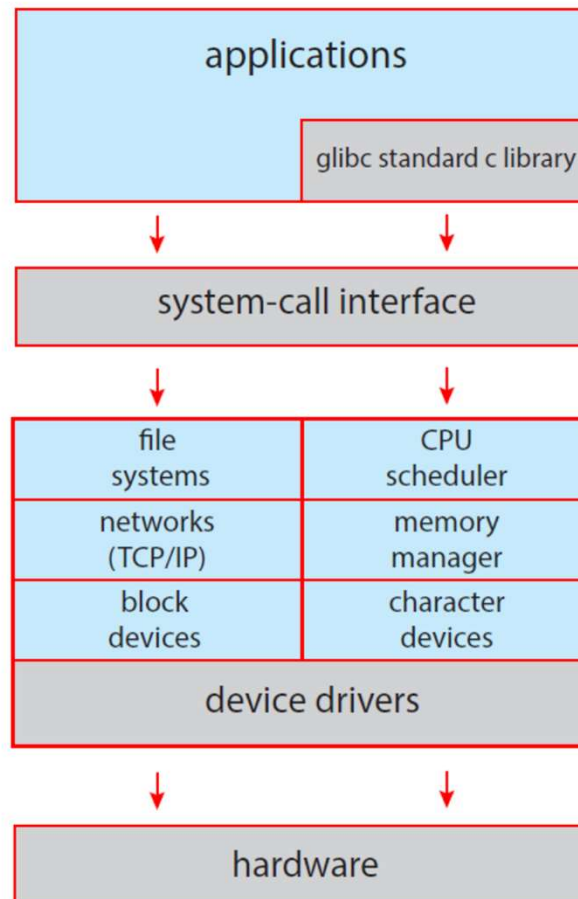
Struttura di un Sistema UNIX

Non semplice



Struttura di un Sistema Linux

Non semplice

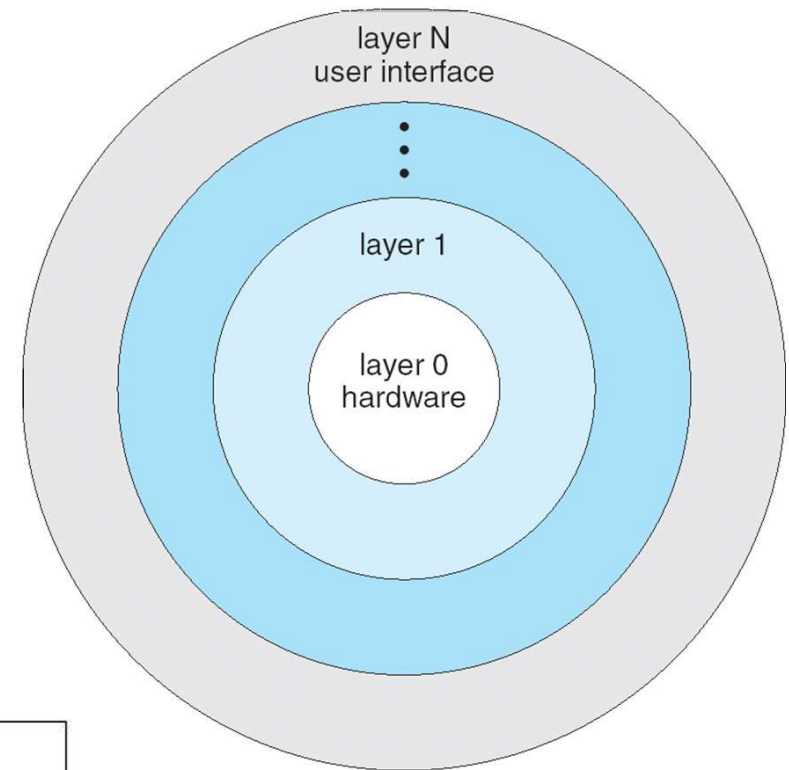


GNU C Library: core lib del GNU system per interagire con il kernel linux

Figure 2.13 Linux system structure.

Approccio Stratificato

- Il sistema operativo è suddiviso in un numero di livelli (levels), ciascuno costruito sopra i livelli inferiori (THE, MULTICS).
- Lo strato inferiore (layer 0), è l'hardware; il più alto (layer N) è l'interfaccia utente.
- Con la modularità ciascuno layer utilizza funzioni (operazioni) e servizi dei layer inferiori

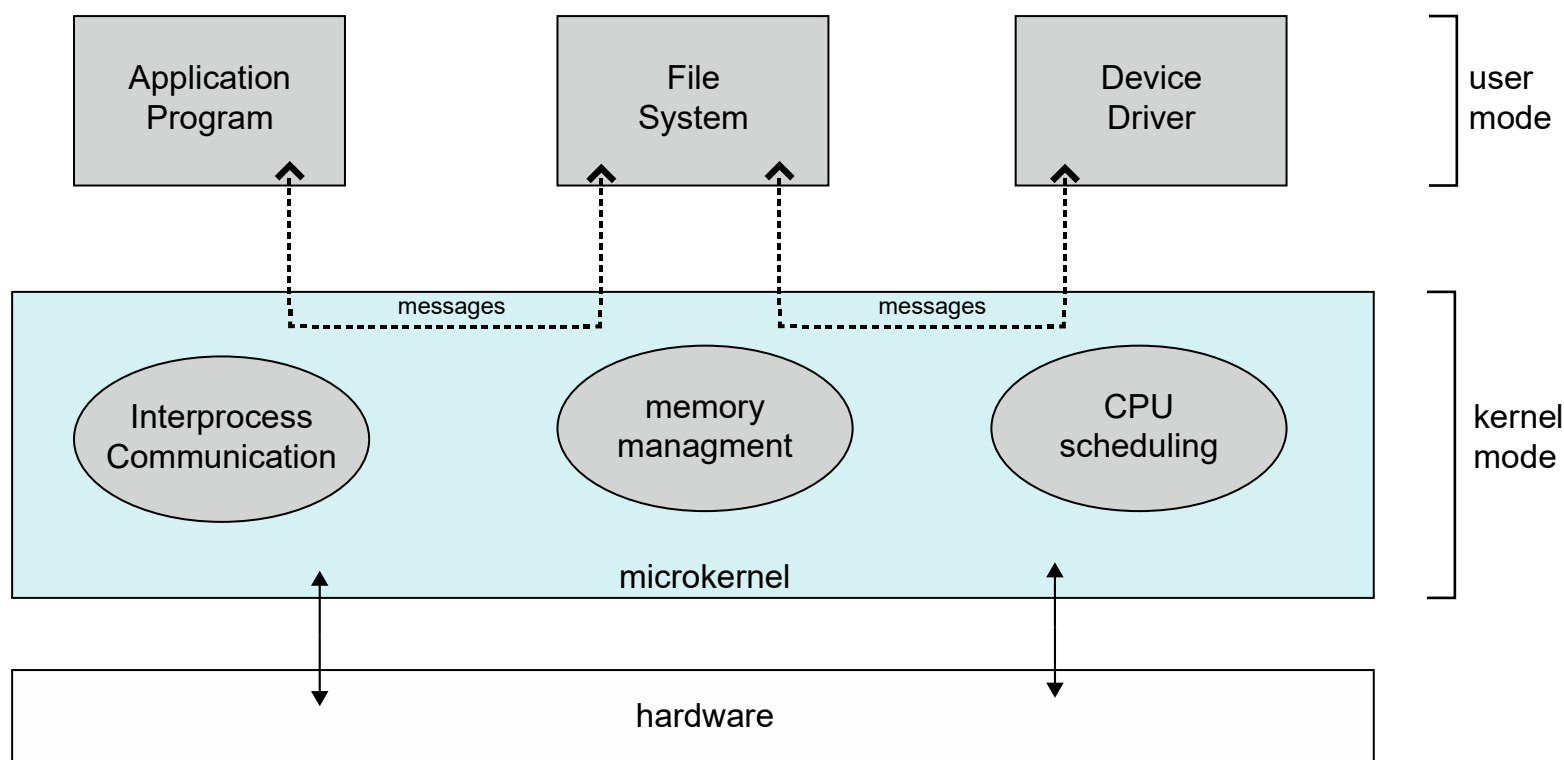


Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

THE realizzato alla Technische Hogeschool Eindhoven in Olanda da Dijkstra nel 1968 per il computer Electrologica X8

Struttura a Microkernel

- ❑ Verso metà anni '80 realizzato **Mach** con kernel strutturato in moduli secondo il cosiddetto **orientamento a microkernel**
 - ❑ macOS kernel (**Darwin**) parzialmente basato su Mach
- ❑ Spostare più possibile servizi dal kernel allo spazio utente
 - ❑ Funzioni di comunicazione tra i programmi client e i vari servizi in esecuzione nello spazio utente – comunicazione message-passer tramite kernel

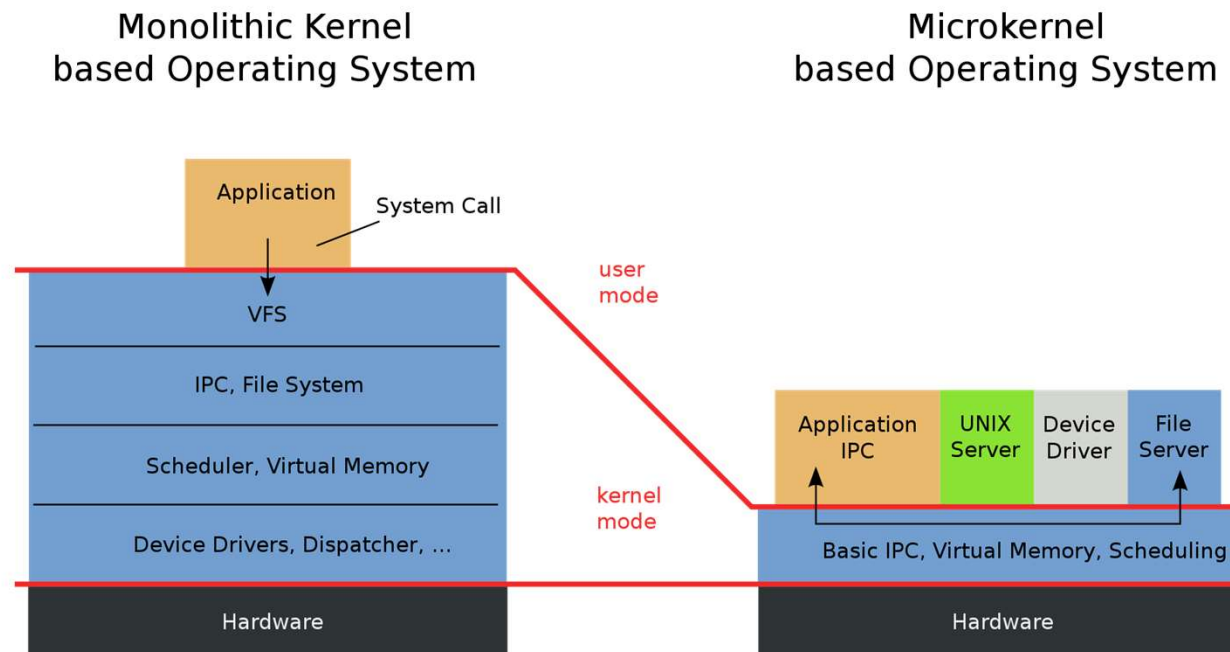


Struttura a Microkernel

- Verso metà anni '80 realizzato **Mach** con kernel strutturato in moduli secondo il cosiddetto **orientamento a microkernel**
 - Mac OS X kernel (**Darwin**) parzialmente basato su Mach
- Spostare più possibile dal kernel allo spazio utente
 - Funzioni di comunicazione tra i programmi client e i vari servizi in esecuzione nello spazio utente
 - Kernel minimale:
 - ▶ meccanismo di comunicazione tra processi (message-passer)
 - ▶ gestione della memoria e dei processi
 - ▶ gestione dell'hardware di basso livello
 - ▶ tutto il resto gestito da processi in spazio utente (e.g., politiche di gestione file system, scheduling, memoria sono implementate da processi utente) che comunicano passando per il kernel

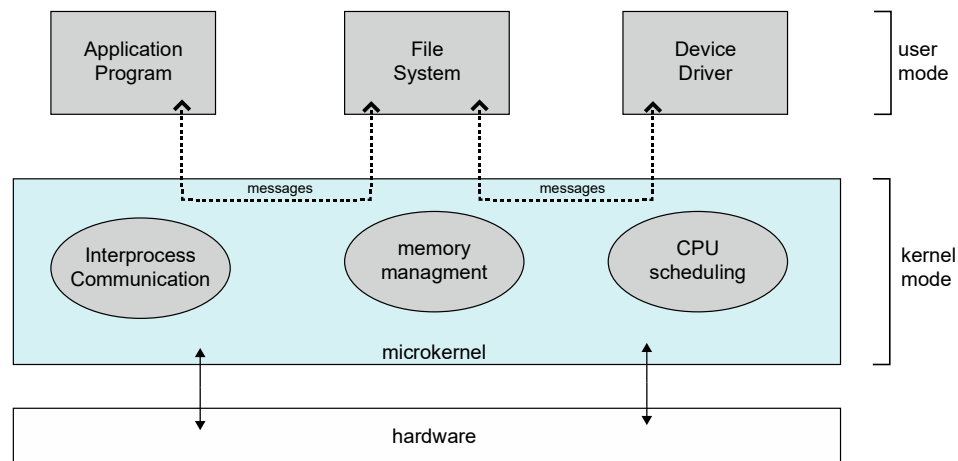
Struttura a Microkernel

- ❑ Verso metà anni '80 realizzato **Mach** con kernel strutturato in moduli secondo il cosiddetto **orientamento a microkernel**
 - ❑ macOS kernel parzialmente basato su Mach (e su BSD)
- ❑ Spostare più possibile dal kernel allo spazio utente
 - ❑ Funzioni di comunicazione tra i programmi client e i vari servizi in esecuzione nello spazio utente
 - ❑ Kernel minimale:



Struttura a Microkernel

- ❑ Spostare più possibile dal kernel allo spazio utente
- ❑ Comunicazione tra moduli utente mediante **message passing**
- ❑ Vantaggi:
 - ❑ Facilità di estensione microkernel
 - ❑ Facilità di trasferimento dell'SO su nuove architetture
 - ❑ Meno codice eseguito in modalità kernel
 - ❑ Sicuro
- ❑ Svantaggi:
 - ❑ Sovraccarico delle prestazioni della comunicazione tra lo spazio utente e lo spazio del kernel (context switch)

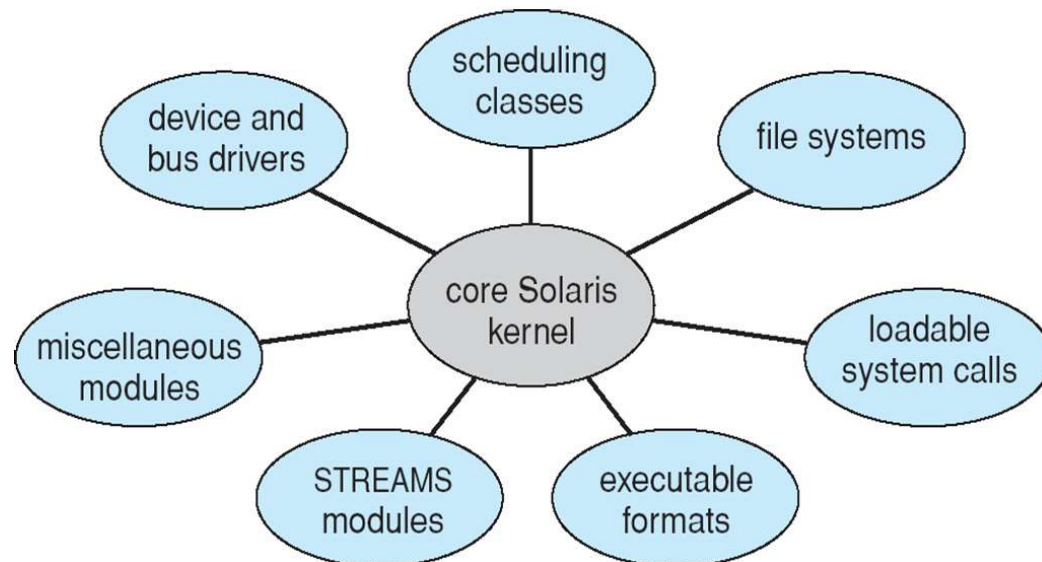


Struttura a Microkernel

- ❑ Caso di Windows NT
 - ❑ Prima release a microkernel
 - ❑ Prestazioni povere rispetto a Windows 95
 - ❑ Si passa a Windows NT 4.0 muovendo servizi da utente a kernel
 - ❑ Windows XP torna più monolitico che microkernel

Moduli

- ❑ Molti sistemi operativi moderni implementano moduli del kernel caricabili
 - ❑ **Loadable kernel modules (LKMs)** - Linux, Solaris, macOS, Win, etc
 - ❑ Ogni componente principale è separato
 - ❑ Servizi linkati dinamicamente mentre il kernel è in esecuzione o caricamento
 - ❑ Ciascuno parla con gli altri tramite interfacce conosciute,
 - ❑ Ciascuno è caricabile secondo necessità all'interno del kernel



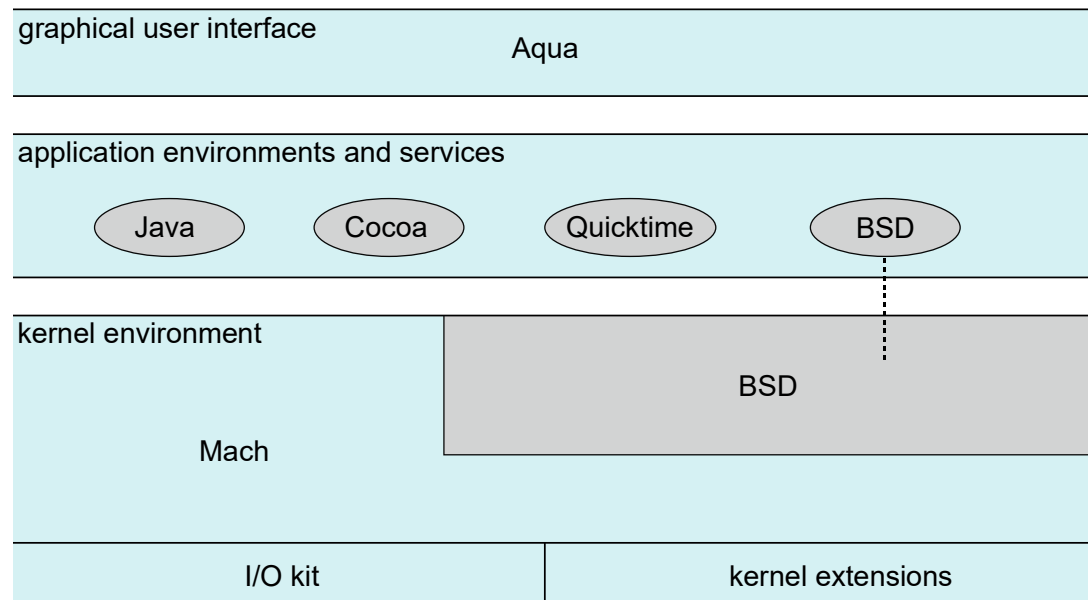
- ❑ Simile al sistema a livelli (core ha interfacce con altri moduli) ma con più flessibilità
- ❑ Simile a microkernel, ma meno message passing

Sistemi Ibridi

- La maggior parte dei sistemi operativi moderni non sono in realtà un modello puro
 - Il sistema ibrido combina più approcci per soddisfare le esigenze di prestazioni, sicurezza e usabilità
 - Kernel Linux e Solaris in un singolo spazio di indirizzi, quindi monolitico, ma modulari per il caricamento dinamico di funzionalità
 - Windows per lo più monolitico, microkernel per diverse parti del sottosistema
 - Apple macOS ibrido (microkernel, monolitico)

Struttura a Ibrida

- Il sistema operativo **macOS** di Apple è progettato per funzionare principalmente su *computer desktop* e *laptop*, mentre **iOS** è un sistema operativo mobile progettato per *iPhone* e *iPad*
- Strato dell'interfaccia utente (**user experience**)
- Strato degli **ambienti applicativi** – sviluppo applicazioni e supporto
- Strato degli **Ambienti di base** – grafica/media
- Strato degli **Ambiente kernel**



Kernel XNU (X is Not Unix)

Combina Mach e BSD
(microkernel e monolitico)

- memoria, IPC, I/O (variante mach)
- protezioni, virtual file system, rete, POSIX (BSD)

Android

- ❑ Sviluppato da Open Handset Alliance (Google) Open Source
- ❑ Basato su Linux kernel (modificato)
- ❑ App in Java (Android API) su Android RunTime (ART) virtual machine o Java Native (JNI)
- ❑ Eseguito su diversi dispositivi: **strato di astrazione hardware** detto **HAL** (*hardware abstraction layer*)
- ❑ Bionic standard C Lib per Android

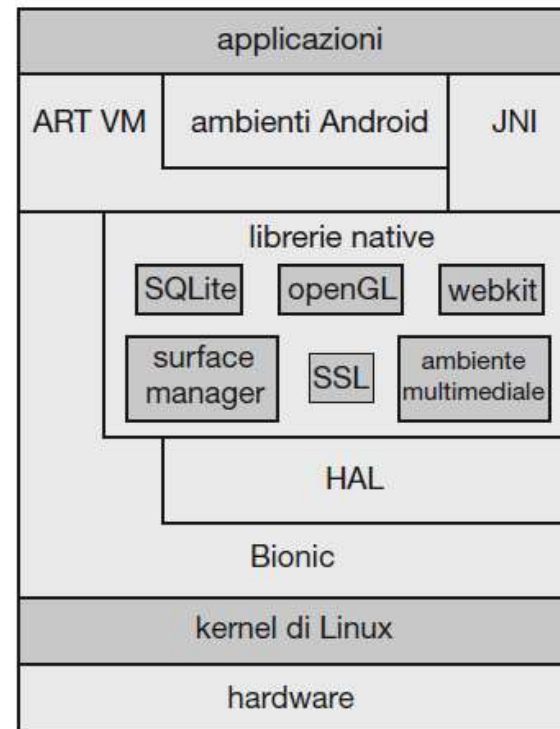


Figura 2.18 Architettura di Google Android.